

Technical Documentation: Integrated Numerical Interpolation Suite

Group: 3

Team Lead: Daksh Agarwal
Team Member 1: Gugulothu Maniprakash
Team Member 2: Maneesh S Kolekar
Team Member 3: Vaibhav Singh
Team Member 4: Samudrala Manish Kumar

Version: 1.0 | Date: February 2, 2026

Abstract

This document serves as the technical specification for the Phase 2 Integrated Interpolation Project. The software is a console-based C++ application designed to perform numerical analysis using five distinct interpolation algorithms: Piecewise Constant, Nearest Neighbor, Linear, Newton’s Divided Difference, and Lagrange Polynomials. The system features a unified input interface, extensive error handling (including boundary value analysis and duplicate detection), and a modular architecture that ensures maintainability and scalability.

Contents

1	System Overview	3
1.1	Purpose and Scope	3
1.2	Software Architecture	3
2	Component Specifications	3
2.1	Global Constants and Data Structures	3
2.2	Validation Modules	3
3	Algorithm Implementation Details	3
3.1	Method 1: Piecewise Constant Interpolation	3
3.1.1	Description	3
3.1.2	Implementation Logic	4
3.2	Method 2: Nearest Neighbor Interpolation	4
3.2.1	Description	4
3.2.2	Implementation Logic	4
3.3	Method 3: Linear Interpolation	4
3.3.1	Description	4
3.3.2	Implementation Logic	4
3.4	Method 4: Newton’s Divided Difference	5
3.4.1	Description	5

3.4.2	Implementation Logic	5
3.5	Method 5: Lagrange Interpolation	5
3.5.1	Description	5
3.5.2	Implementation Logic	5
4	Error Handling and Robustness	5
4.1	Boundary Value Analysis (Extrapolation)	6
4.2	Division by Zero Protection	6
5	Performance Analysis	6
6	Execution Guide	6
6.1	Compilation	6
6.2	Runtime Instructions	6

1 System Overview

1.1 Purpose and Scope

In numerical analysis, interpolation is the process of constructing new data points within the range of a discrete set of known data points. This software automates this process, allowing users to input experimental data (x, y) and obtain estimates for unknown values $f(x)$ using methods varying in complexity from $O(n)$ to $O(n^2)$.

1.2 Software Architecture

The system is built as a monolithic console application using C++98 standards, compatible with Visual Studio 2010. The architecture is procedural and divided into three logical layers:

1. **Input Layer:** Handles user interaction, data ingestion, and rigorous validation (sorting and duplication checks).
2. **Processing Layer:** Contains five isolated functions, each encapsulating a specific mathematical algorithm.
3. **Output Layer:** Formats the floating-point results into a readable tabular format for comparison.

2 Component Specifications

2.1 Global Constants and Data Structures

The system defines specific constraints to ensure memory safety within the stack.

- **MAX_POINTS (50):** A pre-processor constant defining the maximum array size. This prevents buffer overflows during input.
- **double x[], y[]** Standard 1D arrays storing the independent and dependent variables.
- **double diffTable[] []:** A 2D array specific to Newton's method, used to store the divided difference tree.

2.2 Validation Modules

Two helper functions ensure data integrity before calculation begins:

1. **isSorted():** line scans the x array to ensure strict ascending order ($x_i < x_{i+1}$). Required for interval-based methods.
2. **hasDuplicates():** Detects if any $x_i == x_j$. This prevents "Division by Zero" runtime errors in Lagrange and Newton algorithms.

3 Algorithm Implementation Details

This section details the logic, mathematics, and implementation of the five interpolation modules.

3.1 Method 1: Piecewise Constant Interpolation

3.1.1 Description

This is the simplest method, assuming the function value remains constant between data points. It creates a "step" function.

$$f(x) = y_i \quad \text{for} \quad x_i \leq x < x_{i+1}$$

3.1.2 Implementation Logic

The algorithm iterates through the sorted x array. It identifies the interval $[x_i, x_{i+1})$ containing the target x and returns y_i . If the target is out of bounds, an error is logged.

```
1 // Find the interval and return the 'left' value
2 for (int i = 0; i < n - 1; i++) {
3     if (targetX >= x[i] && targetX < x[i + 1]) {
4         return y[i];
5     }
6 }
```

Listing 1: Piecewise Constant Logic

3.2 Method 2: Nearest Neighbor Interpolation

3.2.1 Description

This method selects the value of the data point that is geometrically closest to the target x .

$$f(x) = y_k \quad \text{where} \quad |x - x_k| = \min_j |x - x_j|$$

3.2.2 Implementation Logic

The function performs a linear scan of all points, calculating the absolute difference $|targetX - x_i|$. It tracks the index of the minimum difference found so far.

```
1 double minDiff = abs(targetX - x[0]);
2 for (int i = 1; i < n; i++) {
3     double currentDiff = abs(targetX - x[i]);
4     if (currentDiff < minDiff) {
5         minDiff = currentDiff;
6         nearestIndex = i;
7     }
8 }
```

Listing 2: Nearest Neighbor Logic

3.3 Method 3: Linear Interpolation

3.3.1 Description

This method estimates the value by connecting two adjacent points with a straight line. It is accurate for functions that approximate linearity over small intervals.

$$f(x) = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i}(x - x_i)$$

3.3.2 Implementation Logic

Similar to Piecewise, it locates the interval $[x_i, x_{i+1}]$. Once found, it calculates the slope (m) and applies the point-slope formula.

```
1 if (targetX >= x[i] && targetX <= x[i + 1]) {
2     double slope = (y[i + 1] - y[i]) / (x[i + 1] - x[i]);
3     return y[i] + slope * (targetX - x[i]);
4 }
```

Listing 3: Linear Interpolation Logic

3.4 Method 4: Newton's Divided Difference

3.4.1 Description

This method fits a polynomial of degree $n - 1$ to the data. It uses a recursive table of divided differences to determine polynomial coefficients.

$$P(x) = f[x_0] + (x - x_0)f[x_0, x_1] + \dots$$

3.4.2 Implementation Logic

1. **Table Construction:** A nested loop fills a 2D array where 'table[i][j]' depends on the values of the previous column '[j-1]'. 2. **Evaluation:** A single loop computes the polynomial value using the top row of the table and accumulated product terms.

```
1 for (int j = 1; j < n; j++) {
2     for (int i = 0; i < n - j; i++) {
3         diffTable[i][j] = (diffTable[i+1][j-1] - diffTable[i][j-1])
4                             / (x[i+j] - x[i]);
5     }
6 }
```

Listing 4: Newton's Divided Difference Table Construction

3.5 Method 5: Lagrange Interpolation

3.5.1 Description

Lagrange interpolation finds the polynomial without computing a difference table. It is a linear combination of basis polynomials $L_i(x)$.

$$P(x) = \sum_{i=0}^n y_i L_i(x) \quad \text{where} \quad L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

3.5.2 Implementation Logic

This requires an $O(n^2)$ nested loop structure. The outer loop iterates through terms, and the inner loop calculates the product sequence for the basis polynomial L_i .

```
1 for (int i = 0; i < n; i++) {
2     double term = y[i];
3     for (int j = 0; j < n; j++) {
4         if (i != j) {
5             term = term * (targetX - x[j]) / (x[i] - x[j]);
6         }
7     }
8     result += term;
9 }
```

Listing 5: Lagrange Logic

4 Error Handling and Robustness

To meet Software Engineering quality standards, the following mechanisms are implemented:

4.1 Boundary Value Analysis (Extrapolation)

Methods relying on intervals (Piecewise, Linear) perform a range check at the start of execution.

- **Check:** `if (targetX < x[0] || targetX > x[n-1])`
- **Action:** If true, an error is logged to `stderr`, and a default value is returned to prevent undefined behavior.

4.2 Division by Zero Protection

In Lagrange and Newton methods, the denominators involve differences between x values (e.g., $x_i - x_j$).

- **Check:** The `hasDuplicates()` function scans the input.
- **Action:** If duplicates are found, execution is halted in `main()` before calling the sensitive functions.

5 Performance Analysis

Method	Time Complexity	Space Complexity	Constraint
Piecewise Constant	$O(n)$	$O(n)$	Data must be sorted
Nearest Neighbor	$O(n)$	$O(n)$	None
Linear Interpolation	$O(n)$	$O(n)$	Data must be sorted
Newton's Divided Diff	$O(n^2)$	$O(n^2)$	Requires 2D Table
Lagrange Polynomial	$O(n^2)$	$O(n)$	Computationally heavy

Table 1: Complexity Analysis of implemented methods

Note on Efficiency: While Newton and Lagrange are computationally more expensive ($O(n^2)$), they provide a smooth polynomial curve that passes through all points. Piecewise and Linear methods are faster ($O(n)$) but provide only local approximations.

6 Execution Guide

6.1 Compilation

The source code is contained in a single file `Assignment2.cpp`. Compile using the Visual Studio command line compiler (CL) or the IDE Build button.

```
> cl /EHsc Assignment2.cpp
```

6.2 Runtime Instructions

1. Run the executable `Assignment2.exe`. 2. **Input Count:** Enter the number of points (Integer between 2 and 50). 3. **Input Data:** Enter pairs of numbers separated by space. Ensure X values are increasing.

- Example: `1.0 2.0 [Enter], 3.0 5.0 [Enter]`.

4. **Target:** Enter the X value you wish to interpolate. 5. **Output:** The console will display a formatted table containing the results from all five methods.